



Unión Europea

Fondo Social Europeo

El FSE invierte en tu futuro

PHP

OOP, Object Oriented Programming en PHP

Pepe

INSTITUT



**GENERALITAT
VALENCIANA**

Conselleria d'Educació,
Investigació, Cultura i Esport



GOBIERNO
DE ESPAÑA

MINISTERIO
DE EDUCACIÓN, CULTURA
Y DEPORTE

Continguts

1 Pròleg	3
2 Clases y Objetos	3
3 Herencia	6
3.1 El elemento parent	10
3.2 Convertir un objeto a cadena	10
4 Métodos mágicos	11
5 Espacio de nombres	12
5.1 Acceso	12
6 Excepciones	13
7 Ejercicios	14

1 Pròleg

La **Programación Orientada a Objetos** (OOP, Object Oriented Programming) es un estilo de organizar el código que permite a los desarrolladores agrupar tareas similares en clases. Esto ayuda a que el código sea más fácil de mantener y a no repetirse (**DRY**, Don't Repeat Yourself).

2 Clases y Objetos

Un **clase** es un plantilla que define las propiedades y métodos para poder crear **objetos**. De este manera, un objeto es una instancia de una clase.

- Similar a otros lenguajes como Java o C#
- Representan algo real.
- Se declaran con la palabra reservada **class**
- Entre llaves se declaran los atributos y métodos, que pueden ser privados, públicos o protegidos. Por defecto son públicos
 - **private**: Sólo puede acceder la propia clase.
 - **protected**: Sólo puede acceder la propia clase o sus descendientes.
 - **public**: Puede acceder cualquier otra clase.



Buenas Practicas:

- Todas las clases empiezan por letra mayúscula.
- Siempre definiremos cada clase en un fichero que nombraremos *NombreClase.php*
- Los atributos siempre serán privados o protegidos (crearemos getters y setters para acceder a ellos)

Para poder crear variables u **objetos** de cualquier clase, necesitamos definir una función especial llamada **constructor**. Estas funciones pueden recibir una serie de parámetros, que normalmente son los valores que queremos asignarles a los distintos atributos.

- Para instanciar un objeto a partir de la clase, se utiliza *new*.
- Para acceder desde un objeto a sus atributos o métodos, utilizamos la operador **flecha (->)**
- El objeto **\$this**

- Cuando desde un objeto se invoca un método de la clase, a este se le pasa siempre una referencia al objeto que lo ha llamado. Esta referencia se almacena en la variable **\$this**
- Solo puede haber un **constructor** se llamará **__construct**
 - Será invocada automáticamente al hacer *new*
 - Es ideal para inicializar los datos del objeto antes de usarlo.

Así, podemos definir una clase y un constructor que reciba tres parámetros (uno para el código, otro para el título y otro para la versión) y los asigne a los correspondientes atributos:

```
class Software
{
    private $codigo = 0;
    private $titulo = "";
    private $version = "1";

    public function __construct($c, $t, $v)
    {
        $this->codigo = $c;
        $this->titulo = $t;
        $this->version = $v;
    }

    ...
}
```

Con esto, ya podemos crear variables de este tipo, usando el operador **new** para crear cada objeto. Por ejemplo, así crearíamos dos variables de tipo **Software**, **\$s1** y **\$s2**, con diferentes datos cada una (desde fuera de la clase, si queremos):

```
$s1 = new Software(1, "LibreOffice", "6.0");
$s2 = new Software(2, "GIMP", "3.8");
```

Y para llamar a las funciones de la clase y poder, por ejemplo, mostrar los datos de cada software por pantalla, haríamos algo como esto.

```
echo "<p>Datos del primer programa:</p>";
$s1->MostrarDatos();
echo "<p>Datos del segundo programa:</p>";
$s2->MostrarDatos();
```

Encapsulación Hemos dicho que los atributos de una clase normalmente son privados. Esto es así para no poder acceder a ellos directamente desde fuera, y cambiar su valor erróneamente.

Para cada propiedad, se añaden métodos públicos (getter/setter):

```
public setPropiedad(tipo $param)
public getPropiedad() : tipo
```

Las constantes se definen públicas para que sean accesibles por todos los recursos.

```
class MayorMenor {
    private int $mayor;
    private int $menor;

    public function setMayor(int $may) {
        $this->mayor = $may;
    }

    public function setMenor(int $men) {
        $this->menor = $men;
    }

    public function getMayor() : int {
        return $this->mayor;
    }

    public function getMenor() : int {
        return $this->menor;
    }
}
```

Otra forma de definirlos es con `__get` sirven para obtener el valor del atributo al que representan y `__set` se emplean para modificar el valor del atributo.

```
class Software
{
    private $codigo;
    private $titulo;
    private $version;
    ...

    public function __get($nombre)
    {
        if ($nombre == 'Cod')
            return $this->codigo;
        else if ($nombre == 'Titulo')
            return $this->titulo;
        else if ($nombre == 'Version')
            return $this->version;
    }

    public function __set($nombre, $valor)
    {
        if ($nombre == 'Cod' && $valor > 0)
            $this->codigo = $valor;
        else if ($nombre == 'Titulo')
```

```
        $this->titulo = $valor;  
    else if ($nombre == 'Version')  
        $this->version = $valor;  
    }  
    ...  
}
```

Observa que el método `__get` recibe como parámetro un nombre (el que queramos), y luego dentro de la función emparejamos cada nombre con el atributo al que queramos enlazarlo. Si ponemos `Cod`, lo asociaremos al atributo `$codigo`, y así sucesivamente.

Con esto, si queremos cambiar el código del objeto anterior, podríamos hacerlo con su setter correspondiente, y asegurarnos de que se cambiará a un valor correcto.

```
$s1->Cod = -1; // Esto no hará nada  
$s1->Cod = 10; // Esto sí funcionará
```

La llamada al setter o al getter no es como en el resto de funciones. Tenemos que poner:

- el nombre del objeto.
- el operador `->`.
- y un nombre (de entre los que vaya a aceptar el getter o setter).
- y en el caso de querer asignarle un valor, se lo asignamos como si fuera una variable normal.

Ese nombre, como podemos apreciar, no tiene por qué coincidir con el nombre del atributo. Dentro de la función nos encargamos de emparejar cada nombre con el atributo, así que pueden ser diferentes, ya que la asociación la hacemos nosotros a mano.

3 Herencia

PHP soporta herencia simple, de manera que una clase solo puede heredar de otra, no de dos clases a la vez. Para ello se utiliza la palabra clave `extends`. Si queremos que la clase A hereda de la clase B haremos:

```
class A extends B
```

No es posible extender a múltiples clases, sólo se puede heredar de una clase. Los métodos y propiedades heredados pueden ser sobrescritos declarándolos de nuevo con el mismo nombre que tienen en la clase padre:

```
class Perro  
{
```

```
public $nombre = "Rudolf";
public function ladrar(){
    print "Guau!";
}
}
class Bulldog extends Perro {
    public function ladrar(){
        print "Woof!";
    }
}
$cachorro = new Bulldog(); // Instancia de la clase hija
$cachorro->nombre = "Jeffrey"; // Heredamos la propiedad padre $nombre y
    le asignamos Jeffrey
echo $cachorro->nombre; // Devuelve Jeffrey
$cachorro->ladrar(); // Devuelve Woof! ya que ha sobrescrito la
    función padre ladrar().
```

Ahora que ya tenemos una idea de cómo funciona la herencia de clases, vamos a ver cada uno de los modificadores.

1. Public

Las propiedades y métodos **public** son accesibles desde cualquier parte del script, siendo este modificador el más fácil de usar. En PHP 4 las variables se declaraban con “var” y eran public, pero esa forma de denominarlas está obsoleta y puede generar algunos warnings.

```
class Perro
{
    public $nombre;
    public function ladrar(){
        print "Guau!";
    }
}
class Bulldog extends Perro {
    public function ladrar(){
        print "Woof!";
    }
}
$cachorro = new Bulldog();
$cachorro->nombre = "BunBuns";
print $cachorro->nombre; // Devuelve: BunBuns
```

2. Private

El problema de las propiedades y métodos public es que se permite llamar a los métodos y establecer las propiedades desde cualquier lado del script. En el ejemplo anterior, si en lugar de crear un objeto Bulldog, creamos un objeto Perro e intentamos establecer nombre, nos dará un fatal error: Cannot access private property.

```
class Perro
{
    private $nombre;
    public function ladrar(){
        print "Guau!";
    }
}
$cachorro = new Perro();
$cachorro->nombre = "BunBuns"; // Fatal error
```

No se podrá ni mostrar ni modificar el nombre. Para hacerlo se utilizan getters y setters, métodos public para poder acceder a estas propiedades y métodos private desde otras partes del script:

```
class Perro
{
    private $nombre;
    public function ladrar(){
        print "Guau!";
    }
    public function setNombre($nombre){
        $this->nombre = $nombre;
    }
    public function getNombre(){
        return $this->nombre;
    }
}
$cachorro = new Perro();
$cachorro->setNombre("Chicken");
echo $cachorro->getNombre(); // Devuelve: Chicken
```

3. Protected

Las propiedades y métodos marcados como **protected** son accesibles a través de la clase donde se crean y sus descendientes:

```
class Perro
{
    protected $nombre;
    protected function ladrar(){
        print "Guau!";
    }
}
class Bulldog extends Perro {
    public function ladrarBulldog(){
        // Podemos acceder a ladrar() de la clase Perro
        return $this->ladrar();
    }
    public function setNombre($nombre){
        $this->nombre = $nombre;
    }
}
```



```
}  
public function getNombre(){  
    print $this->nombre;  
}  
}
```

Diferentes ejemplos de acceso a propiedades **protected**:

```
$cachorro = new Bulldog();  
// Podemos acceder a la función ladrar() desde ladrarBulldog():  
$cachorro->ladrarBulldog(); // Devuelve Guau!  
// No podemos acceder a ladrar() desde el objeto cachorro:  
$cachorro->ladrar(); // Fatal error: Call to protected method Perro::  
    ladrar()  
// Tampoco podemos asignarle un nombre, pues $nombre también es protected  
$cachorro->nombre = "Hunky"; // Fatal Error: Cannot access protected  
    property  
// Si podemos asignarle un valor con el método setNombre():  
$cachorro->setNombre("Hunky");  
// Y mostrarlo con getNombre():  
echo $cachorro->getNombre(); // Devuelve: Hunky
```

5. Abstract

El modificador **abstract** indica que una clase o método no puede instanciarse y sólo puede heredarse, trasladando su funcionamiento obligatorio a las clases hijas.

```
abstract class CocheAbstract {  
  
    public function getRuedas()  
    {  
        return 4;  
    }  
    abstract public function setPotencia($potencia);  
    abstract public function getPotencia();  
}  
class Audi extends CocheAbstract {  
    public $brand = 'Audi';  
    protected $potencia;  
    public function setPotencia($potencia)  
    {  
        $this->potencia = $potencia;  
    }  
    public function getPotencia()  
    {  
        return $this->potencia;  
    }  
}  
$audi = new Audi;  
$ruedas = $audi->getRuedas();
```

```
$audi->setPotencia(100);  
$potencia = $audi->getPotencia();  
  
echo "Coche " . $audi->brand . " de " . $ruedas . " ruedas " . "y " .  
    $potencia . " cv de potencia";  
// Devuelve Coche Audi de 4 ruedas y 100 cv de potencia
```

El método `getRuedas()` será igual para todas las marcas de coches, por eso se define en la clase abstracta. La clase Audi tendrá ese método para usar, y además deberá definir los métodos `setPotencia()` y `getPotencia()`.

3.1 El elemento parent

Hemos visto que el objeto `$this` nos sirve para referenciar a los atributos o elementos de una clase, y poderlos distinguir de otros externos que se llamen igual. Del mismo modo, podemos referenciar a los atributos o métodos de la clase padre mediante el objeto `parent`, con la sintaxis:

```
parent::metodo(...)
```

super() en `java`. Así, por ejemplo, no sería necesario duplicar el código en el constructor de la clase hija para los atributos que asigna la clase padre. Podríamos poner:

```
public function __construct($c, $t, $v, $p)  
{  
    parent::__construct($c, $t, $v);  
    $this->plataforma = $p;  
}
```

3.2 Convertir un objeto a cadena

Otra de los métodos mágicos interesantes es el método especial denominado `__toString()` que devolverá una cadena representando el estado actual de un objeto.

Si en una clase sobrescribimos el método, éste se invocará al intentar imprimir la clase por ejemplo, haciendo **echo \$objeto**.

```
class Persona {  
    protected $dni;  
    protected $nombre;  
  
    public function __construct($pDni, $pNombre) {  
        //constructor  
  
        $this->dni = $pDni;  
        $this->nombre = $pNombre;  
    }  
}
```

```
    }

    public function __toString()
    {
        $texto = 'DNI: '.$this->dni.'<br>';
        $texto .= 'Nombre: '.$this->nombre . '<br>';
        return $texto;
    }
}
$alumno = new Persona('12345678', 'Pepe Devesa');
echo $alumno; //Llamada implícita al método __toString()
?>
```

Al ejecutar ese código tendremos como resultado lo siguiente:

```
DNI:12345678
Nombre: Pepe Devesa
```

4 Métodos mágicos

Todas las clases PHP ofrecen un conjunto de métodos, también conocidos como **magic methods** que se pueden sobrescribir para sustituir su comportamiento. Algunos de ellos ya los hemos utilizado.

Ante cualquier duda, es conveniente consultar la [documentación oficial](#).

Los más destacables son:

- `__construct()`
- `__destruct()` → se invoca al perder la referencia. Se utiliza para cerrar una conexión a la BD, cerrar un fichero, ...
- `__toString()` → representación del objeto como cadena. Es decir, cuando hacemos `echo $objeto` se ejecuta automáticamente este método `__get(propiedad)`, `__set(propiedad, valor)` → Permitiría acceder a las propiedad privadas, aunque siempre es más legible/mantenible codificar los getter/setter.
- `__isset(propiedad)`, `__unset(propiedad)` → Permite averiguar o quitar el valor a una propiedad.
- `__sleep()`, `__wakeup()` → Se ejecutan al recuperar (`unserialize()`) o almacenar un objeto que se serializa (`serialize()`), y se utilizan para permite definir qué propiedades se serializan.
- `__call()`, `__callStatic()` → Se ejecutan al llamar a un método que no es público. Permiten sobrecargar métodos.

5 Espacio de nombres

También conocidos como **Namespaces**, permiten organizar las clases/interfaces, funciones y/o constantes de forma similar a los paquetes en Java. Se declaran en la primera línea mediante la palabra clave **namespace** seguida del nombre del espacio de nombres asignado (cada subnivel se separa con la barra invertida \):

Por ejemplo, para colocar la clase Producto dentro del **namespace dwes** lo haríamos así:

```
namespace dwes\Ejemplos;

const IVA = 0.21;

class Producto {
    public $nombre;

    public function muestra() : void {
        echo"<p>Prod:" . $this->nombre . "</p>";
    }
}
```

5.1 Acceso

Para referenciar a un recurso que contiene un namespace, primero hemos de tenerlo disponible haciendo uso de `include` o `require`. Si el recurso está en el mismo namespace, se realiza un acceso directo (se conoce como acceso sin cualificar).

Realmente hay tres tipos de acceso:

- sin cualificar: recurso
- cualificado: rutaRelativa\recurso → no hace falta poner el namespace completo
- totalmente cualificado: \rutaAbsoluta\recurso

```
namespace Dwes\Ejemplos;

include_once("Producto.php");

echo IVA; // sin cualificar
echo Utilidades\IVA; // acceso cualificado. Daría error, no existe \Dwes\Ejemplos\Utilidades\IVA
echo \Dwes\Ejemplos\IVA; // totalmente cualificado

$p1 = new Producto(); // lo busca en el mismo namespace y encuentra \Dwes\Ejemplos\Producto
```

```
$p2 = new Model\Producto(); // daría error, no existe el namespace Model.  
      Está buscando \Dwes\Ejemplos\Model\Producto  
$p3 = new \Dwes\Ejemplos\Producto(); // \Dwes\Ejemplos\Producto
```

6 Excepciones

- Exception es la clase base para todas las excepciones de usuario en PHP.
- Proporcionan métodos para obtener información de la excepción y de traza
 - getMessage. Devuelve el mensaje, en caso de que se haya puesto algún
 - getCode. Devuelve el código de error si existe

Exempciones definidas por el usuario Otra opción para indicar un error es lanzar una exención.

- El código susceptible de producir algún error se introduce en un bloque **try**
- Cuando se produce algún error, se lanza una excepción utilizando la instrucción **throw**
- Después del bloque try tiene que haber como mínimo un bloque **catch** encargado de procesar el error
- Si una vez acabado el bloque try no se ha lanzado ninguna excepción, se continúa con la ejecución en la línea siguiente al bloque o bloques catch
- Si hay algo que se tenga que ejecutar tanto si se produce una excepción como si no se produce, lo pondremos dentro de un bloque **finally**, después del último bloque catch.
- Para lanzar una excepción no es necesario indicar ningún parámetro, aunque de forma opcional se puede pasar un mensaje de error y también un código de error

```
class DivisionByZero extends exception{  
    protected $message = "El segundo argumento es 0";  
}  
  
function dividir($a, $b){  
    if ($b==0){  
        throw new DivisionByZero;  
    }  
    return $a/$b;  
}  
try{  
    $resul1 = dividir(5, 0);  
    echo "Resul 1 $resul1". "<br>";  
}catch(DivisionByZero $e){  
    echo "Excepción: ". $e->getMessage(). "<br>";  
}finally{  
    echo "Primer finally<br>";  
}
```

```
}  
  
try{  
    $resul2 = dividir(5, 2);  
    echo "Resul 2 $resul2". "<br>";  
}catch(DivisionByZero $e){  
    echo "Excepción: ". $e->getMessage(). "<br>";  
}finally{  
    echo "Segundo finally";  
}
```

7 Ejercicios

7.0.1 Clases.php

Crea una página llamada clases.php con:

- una clase llamada **Persona** que tenga como atributos un DNI, un nombre y un email.
 - Crea un constructor que permita rellenar esos tres atributos,
 - y los getters y setters correspondientes.
 - Define también un método Mostrar para sacar por la página los datos de la persona (en un párrafo, separados por guiones).
 - Define adecuadamente la visibilidad (pública o privada) de cada atributo o método.
- Crea una segunda clase llamada Estudiante que:
 - herede de Persona, y
 - añada un atributo llamado numExpediente.
 - Crea su constructor, sus getters y setters y su correspondiente método Mostrar.

Fuera de las clases, entre el código HTML de la página, crea un objeto de cada tipo (una Persona y un Estudiante), con los valores que quieras, llama después a algún setter de cada una para cambiar el valor de algún atributo, y finalmente llama a sus métodos Mostrar para que saquen la información de cada uno.

7.0.2 Player.php

Crea un clase **Player**, con los campos:

- Name
- BirthDay
- Country
- Dorsal
- Position
- Goals
- Matches
- Minutes
- YellowCard
- RedCard

Crea los métodos:

- Age()
- Score()
- AddCard(int Colour)
- PlayMinutes(int min)
- Render() para mostrar la ficha del jugador
- Construct()

Crea un clase **Team**, amb el següents camps:

- Name
- Players
- Matches
- Won
- Lost
- Tie
- ScoreGoals
- ConcededGoals

Crea los métodos:

- Construct()
- Render() mostrar el equipo
- SignPlayer(Player)

Importa el fichero y cargalo el Team de l'Atletic de Madrit y muestra la palntilla en pantalla utilizando la función render de la clase Team.

Team: Atlético de Madrid
 Matches: 0
 Won: 0
 Lost: 0
 Tie: 0
 Score Goals: 0
 Conceded Goals: 0
 Players:

Player	Age	Country	Dorsal	Position	Goals	Matches	Minutes	Yellow Cards	Red Cards
Juan Carlos Aguilera Martín	55	España	LTD	15	10	31	10	1834	2
Radek Bejbl	51	República Checa	MED	24	4	33	19	2472	2
Leonardo Ángel Biagini	47	Argentina	DEC	7	15	22	3	805	1
José Luis Pérez Caminero	56	España	MED	21	3	30	18	2303	14
Juan Eduardo Esnáider Belén	51	Argentina	DEL	9	1	35	23	2833	16
Santiago Ezquerro Marín	47	España	DEL	28	1	4	0	201	0
Quinton Fortune	47	Sudáfrica	MED	26	2	2	0	73	0
Roberto Luis Fresnedoso Prieto	51	España	MED	18	19	27	5	1014	2
Delfi Geli Roura	55	España	LTD	20	1	30	22	2375	1
Iván Rocha Lima	55	Brasil	CEN	25	0	0	0	0	0
Iván Rodríguez Rabell	46	España	LTI	30	1	1	0	5	0

(a) Team

7.0.3 class.articulo.php

Crea dos clases, **Articulo** y **ArticuloRebajado** que será una subclase de **Articulo**.

- La clase **Articulo** se guardará en un fichero llamado “class.articulo.php” y contendrá dos atributos protegidos **nombre** y **precio**
- El constructor recibirá como parámetros obligatorios pNombre y pPrecio y le asignará dichos valores a sus correspondientes atributos.
- Implementar el método __toString() que devuelva la siguiente cadena:

```
'Nombre:' . $this->nombre . '<br />' . 'Precio: ' . $this->precio . '&euro'
; <br />" ;
```

(usa el operador . =)

- Un método **getPrecio** que devuelva el valor actual del precio y **setPrecio(\$pPrecio)** que compruebe si la variable \$pPrecio es un valor numérico y si es así, se lo asigne al atributo **precio** de la clase **Articulo**.

Crea un nuevo fichero llamado “herencia.php” que requiera la definición (usa la función que comprueba que no se haya requerido la definición de la clase anteriormente y si es así provoca un error fatal)

```
if (!class_exists('Articulo'))
```

“Articulo” y dentro de dicho fichero crea la clase **ArticuloRebajado** que será declarada como **final** (no se podrá heredar de ella) y será hija de la clase **Articulo**.

Esta clase contendrá:

- Un atributo privado llamado **\$rebaja**
- El constructor por defecto que recibe como parámetros obligatorios \$pNombre, \$pPrecio y \$pRebaja. Dentro de dicho constructor habrá que llamar al constructor del padre para darle valor a nombre y precio. Después le daremos valor al atributo rebaja.
- Un método privado llamado **calculaDescuento()** que nos devuelve el precio por la rebaja dividido por 100.
- Un método público llamado **precioRebajado()** que nos devuelve la diferencia entre el precio y el descuento.
- El método **__toString()** que nos devuelve:
 - Lo que nos devuelve el método **__toString()** de la clase padre **Articulo** (habrá que realizar una llamada explícita al método **__toString()** del padre) Junto con la cadena 'La rebaja es:'. `$this->rebaja . ' %';` y por último 'El descuento es: `self::calculaDescuento()`. '€"

Después de definir la clase **ArticuloRebajado** crea una instancia de la misma con los parámetros:

```
nombre='Bicicleta', precio=352.10 y rebaja=20.
```

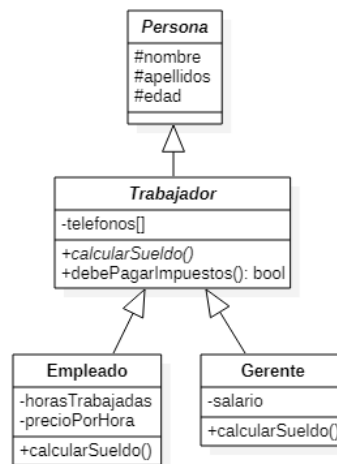
Imprime con un echo el objeto y en la siguiente línea esta cadena “*El precio del artículo rebajado es*” concatenado con la función que nos devuelve el valor del precio rebajado junto el signo del euro y un salto de línea. Debajo de esa línea muestra en formato preformateado () lo que devuelve la función `var_dump()` del objeto creado anteriormente.

El resultado tiene que ser el siguiente:

```
Nombre: Bicicleta
Precio: 352.10
La rebaja es: 20%
El descuento es: 70.42
El precio del articulo rebajado es 281.68
object(ArticuloRebajado)[1]
  private 'rebaja' => int 20
  protected 'nombre' => string 'Bicicleta' (length=9)
  protected 'precio' => float 352.1
```

7.0.4 Trabajadores

Basandose en el UML siguiente



(b) Trabajadores

Crear el desarrollo en php teniendo en cuenta:

- **Persona** es **clase abstracta** donde su método estático `toHtml(Persona $p)` tenga que ser redefinido en todos sus hijos.

```

public static function toHtml(Persona $p): string {
    if ($p instanceof Empleado) {
        // Aquí ya podemos acceder a las propiedades y métodos de Empleado
    }
}
  
```

- **Trabajador** es una **clase abstracta** que almacena los *telefonos* y donde *calcularSueldo* es un método abstracto de manera que:
 - El sueldo de un **Empleado** se calcula a partir de las horas trabajadas y lo que cobra por hora.
 - Para los **Gerentes**, su sueldo se incrementa porcentualmente en base a su edad: $salario + \frac{salario \cdot edad}{100}$
 - A la hora de saber si un empleado debe pagar impuestos, lo hará siempre y cuando tenga más de 21 años y dependa del valor de su sueldo.